

3D ULPIN: Giving Every Vertical Property an Identity

Project technical documentation | Prototype implementation

****Smart India Hackathon | Prototype explanation in six slides****

Slide 1: The Problem

Land records are mostly flat

- A normal land record identifies the ground parcel.
- It does not clearly identify each floor, apartment, basement, or underground utility space.
- City data is often spread across maps, registers, tax records, and survey files.
- Officials cannot easily see the full property in one place.

****Simple example:**** One plot may contain a shop on the ground floor, four homes above it, and a basement below it. A single flat parcel number does not describe all these units.

Slide 2: The Real-Life Need

Why this matters to cities

Municipal and land-record teams need to know:

- What is built on each parcel?
- How many floors or units exist?
- Is any building part outside the legal boundary?
- Which property should be checked for tax or planning purposes?
- Where are buildings and underground services located?

Without one clear 3D record, checking this information is slow and can lead to missing or outdated records.

****Users:**** municipal corporations, land departments, survey teams, planners, tax teams, and emergency services.

Slide 3: Our Solution

One parcel record, including its vertical units

Our system creates a digital record for each parcel.

It connects:

- the parcel boundary;
- the building footprint;
- each floor or vertical unit;
- basement and underground units;
- map location, area, and land-use information;
- a unique prototype 3D ULPIN.

The base ULPIN identifies the parcel. A suffix identifies the level:

- -F00: ground floor;
- -F01: first floor;
- -B01: first basement;
- -UTL: underground utility space.

The user can search the parcel, view it on a map, open it in 3D, and check possible boundary encroachment.

Slide 4: How the Prototype Works

Simple workflow

1. Select a city ward
|
v
2. Load the ward boundary
|
v
3. Get building footprints from OpenStreetMap
or create test parcels when data is unavailable
|
v
4. Create parcel records and calculate area
|
v
5. Generate a ULPIN for each parcel
|
v
6. Give buildings height and split them into floors
|
v
7. Check building boundary overlap
and create a 3D point-cloud view
|
v
8. Save the result and show it in the web application

****Result:**** a searchable 2D map, a 3D building view, floor-level IDs, and simple spatial checks.

Slide 5: Existing Solutions and Why Ours Helps

Existing solutions

- **Paper and spreadsheets:** simple, but difficult to search and update.
- **2D GIS portals:** good for boundaries, but often do not identify floors and basements.
- **Drone, survey, and LiDAR data:** useful measurements, but data alone is not a connected property system.
- **Large digital-twin platforms:** powerful, but expensive and broader than this focused problem.

Why our prototype is a good approach

- It starts with open map data and common formats.
- It links the ground parcel to its vertical units.
- It shows the result in both 2D and 3D.
- It can flag a possible building encroachment.
- It is modular: better survey data can replace test data later.
- It is low-cost and suitable for a municipal pilot.

It does not try to replace a legal survey. It gives officials one useful spatial view for checking and connecting records.

Slide 6: Technology and Prototype Boundary

What each main technology does

- **Python:** runs the main application logic.
- **FastAPI:** provides the web API used by the frontend.
- **OpenStreetMap (OSM):** supplies building footprints and basic map information.
- **Shapely:** checks and compares shapes such as parcels and buildings.
- **pyproj:** converts map coordinates into metre-based coordinates for accurate area and distance calculations.
- **PostgreSQL/PostGIS:** planned production spatial database. SQLite is used as the local fallback in this prototype.
- **LiDAR:** represents ground, buildings, and vegetation as 3D points. The current demo generates synthetic LiDAR points.
- **Three.js/browser frontend:** displays parcels, buildings, floors, and 3D meshes.
- **Docker Compose:** starts the PostGIS database and map-tile service for deployment.

What the prototype proves

It proves that a ward can be converted into searchable parcels with 3D buildings, floor-level IDs, map output, and basic checks through one application.

What production would add

Official cadastral boundaries, measured LiDAR and elevation data, verified ownership records, official ULPIN compliance, user permissions, audit history, and municipal data validation.